



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 06

NOMBRE COMPLETO: VELARDE VALENCIA JOSUE

N° de Cuenta: 320342330

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 08/10/2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

Para el primer ejercicio modifiqué la imagen proporcionada por el profesor de los logos de Star Wars en GIMP para incluir solo los que usaría en una imagen de 1024x1024, exportando en .jpeg y en .tga, siendo la siguiente:



Exporté la textura a OpenGL, usando el archivo jpeg:

```
Texture holocronTexture;  
holocronTexture = Texture("Textures/LogosStarWars.jpeg");  
holocronTexture.LoadTextureA();
```

Con ayuda de GeoGebra obtuve uno por uno los ejes de cada imagen, para colocarlos en los valores S y T del modelo del holocrón creado por mí, diferente al proporcionado en la práctica.

Para esto, hice la definición de los índices correspondientes.

```

void CrearHolocron()
{
    unsigned int holocron_indices[] = {
        // Frente
        0, 1, 2,
        2, 3, 0,
        // Lateral derecho
        4, 5, 6,
        6, 7, 4,
        // Parte posterior
        8, 9, 10,
        10, 11, 8,
        // Lateral izquierdo
        12, 13, 14,
        14, 15, 12,
        // Base inferior
        16, 17, 18,
        18, 19, 16,
        // Base superior
        20, 21, 22,
        22, 23, 20,
        // Esquinas superiores
        24, 25, 26,
        27, 28, 29,
        // Esquinas inferiores
        30, 31, 32,
        33, 34, 35,
        // Esquinas posteriores superiores
        36, 37, 38,
        39, 40, 41,
        // Esquinas posteriores inferiores
        42, 43, 44,
        45, 46, 47,
    };
};

```

Y posteriormente, como ya mencioné, creé y realicé el texturizado de cada cara del holocrón.

```

GLfloat holocron_vertices[] = {
    // Frente
    // X    Y    Z        S    T        NX    NY    NZ
    -0.5f, 0.0f, 0.5f,  0.06f, 0.76f,  0.0f, 0.0f, -1.0f,
    0.0f, 0.5f, 0.5f,  0.06f, 0.96f,  0.0f, 0.0f, -1.0f,
    0.5f, 0.0f, 0.5f,  0.24f, 0.96f,  0.0f, 0.0f, -1.0f,
    0.0f, -0.5f, 0.5f,  0.24f, 0.76f,  0.0f, 0.0f, -1.0f,
    // Derecha
    0.5f, 0.0f, -0.5f,  0.3f, 0.76f, -1.0f, 0.0f, 0.0f,
    0.5f, 0.5f, 0.0f,  0.3f, 0.96f, -1.0f, 0.0f, 0.0f,
    0.5f, 0.0f, 0.5f,  0.46f, 0.96f, -1.0f, 0.0f, 0.0f,
    0.5f, -0.5f, 0.0f,  0.46f, 0.76f, -1.0f, 0.0f, 0.0f,
    // Atrás
    -0.5f, 0.0f, -0.5f, 0.54f, 0.76f,  0.0f, 0.0f, 1.0f,
    0.0f, 0.5f, -0.5f, 0.54f, 0.96f,  0.0f, 0.0f, 1.0f,
    0.5f, 0.0f, -0.5f, 0.7f, 0.96f,  0.0f, 0.0f, 1.0f,
    0.0f, -0.5f, -0.5f, 0.7f, 0.76f,  0.0f, 0.0f, 1.0f,
    // Izquierda
    -0.5f, 0.0f, 0.5f,  0.78f, 0.76f, -1.0f, 0.0f, 0.0f,
    -0.5f, 0.5f, 0.0f,  0.78f, 0.96f, -1.0f, 0.0f, 0.0f,
    -0.5f, 0.0f, -0.5f, 0.94f, 0.96f, -1.0f, 0.0f, 0.0f,
    -0.5f, -0.5f, 0.0f, 0.94f, 0.76f, -1.0f, 0.0f, 0.0f,
    // Inferior
    0.5f, -0.5f, 0.0f,  0.14f, 0.52f,  0.0f, 1.0f, 0.0f,
    0.0f, -0.5f, -0.5f, 0.14f, 0.72f,  0.0f, 1.0f, 0.0f,
    -0.5f, -0.5f, 0.0f, 0.3f, 0.72f,  0.0f, 1.0f, 0.0f,
    0.0f, -0.5f, 0.5f, 0.3f, 0.52f,  0.0f, 1.0f, 0.0f,
    // Superior
    0.5f, 0.5f, 0.0f,  0.53f, 0.04f,  0.0f, -1.0f, 0.0f,
    0.0f, 0.5f, -0.5f, 0.53f, 0.24f,  0.0f, -1.0f, 0.0f,
    -0.5f, 0.5f, 0.0f, 0.7f, 0.24f,  0.0f, -1.0f, 0.0f,
    0.0f, 0.5f, 0.5f, 0.7f, 0.04f,  0.0f, -1.0f, 0.0f,
};

```

```

// Frente superior derecho
0.5f, 0.0f, 0.5f, 0.56f, 0.5f, 0.577f, 0.577f, 0.577f,
0.5f, 0.5f, 0.0f, 0.4f, 0.71f, 0.577f, 0.577f, 0.577f,
0.0f, 0.5f, 0.5f, 0.7f, 0.71f, 0.577f, 0.577f, 0.577f,
// Frente superior izquierdo
-0.5f, 0.5f, 0.0f, 0.13f, 0.2f, -0.577f, 0.577f, 0.577f,
-0.5f, 0.0f, 0.5f, 0.0f, 0.47f, -0.577f, 0.577f, 0.577f,
0.0f, 0.5f, 0.5f, 0.25f, 0.47f, -0.577f, 0.577f, 0.577f,
// Frente inferior derecho
0.5f, -0.5f, 0.0f, 0.64f, 0.72f, 0.577f, -0.577f, 0.577f,
0.5f, 0.0f, 0.5f, 0.8f, 0.5f, 0.577f, -0.577f, 0.577f,
0.0f, -0.5f, 0.5f, 0.92f, 0.72f, 0.577f, -0.577f, 0.577f,
// Frente inferior izquierdo
-0.5f, 0.0f, 0.5f, 0.38f, 0.22f, -0.577f, -0.577f, 0.577f,
-0.5f, -0.5f, 0.0f, 0.25f, 0.47f, -0.577f, -0.577f, 0.577f,
0.0f, -0.5f, 0.5f, 0.5f, 0.47f, -0.577f, -0.577f, 0.577f,
// Atrás superior derecho
0.5f, 0.5f, 0.0f, 0.49f, 0.46f, 0.577f, 0.577f, -0.577f,
0.5f, 0.0f, -0.5f, 0.61f, 0.26f, 0.577f, 0.577f, -0.577f,
0.0f, 0.5f, -0.5f, 0.71f, 0.46f, 0.577f, 0.577f, -0.577f,
// Atrás superior izquierdo
-0.5f, 0.0f, -0.5f, 0.0f, 0.22f, -0.577f, 0.577f, -0.577f,
-0.5f, 0.5f, 0.0f, 0.12f, 0.0f, -0.577f, 0.577f, -0.577f,
0.0f, 0.5f, -0.5f, 0.25f, 0.22f, -0.577f, 0.577f, -0.577f,
// Atrás inferior derecho
0.5f, 0.0f, -0.5f, 0.38f, 0.29f, 0.577f, -0.577f, -0.577f,
0.5f, -0.5f, 0.0f, 0.51f, 0.03f, 0.577f, -0.577f, -0.577f,
0.0f, -0.5f, -0.5f, 0.24f, 0.03f, 0.577f, -0.577f, -0.577f,
// Atrás inferior izquierdo
-0.5f, -0.5f, 0.0f, 0.72f, 0.28f, -0.577f, -0.577f, -0.577f,
-0.5f, 0.0f, -0.5f, 0.84f, 0.54f, -0.577f, -0.577f, -0.577f,
0.0f, -0.5f, -0.5f, 0.98f, 0.28f, -0.577f, -0.577f, -0.577f,

```

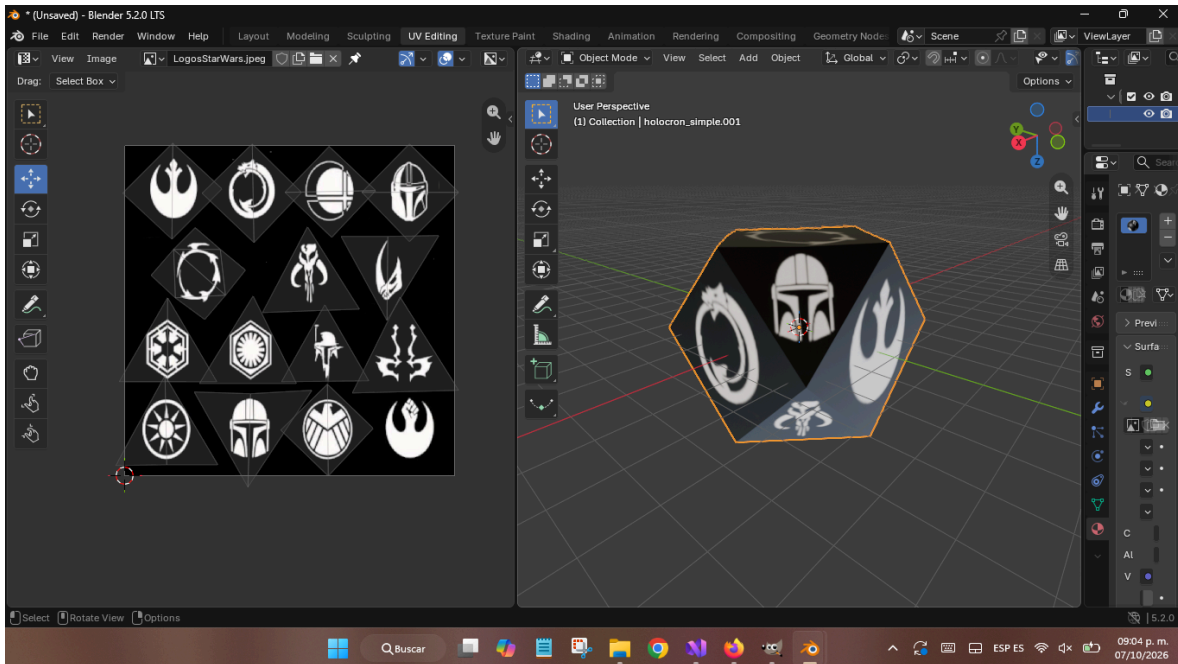
Y dibujo el modelo con un color verde para resaltar los logos y diferenciarlo del otro holocrón.

```

//Holocron Código
color = glm::vec3(0.0f, 1.0f, 0.0f); //verde
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 5.0f, 0.0f));
model = glm::scale(model, glm::vec3(4.5f, 4.5f, 4.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
holocronTexture.UseTexture();
meshListModel[5] -> RenderMeshModel();

```

La siguiente actividad requirió el uso de Blender para realizar el texturizado, a partir del modelo holocron_simple.obj de la práctica, texturicé con la imagen realizada para la actividad anterior, sin seguir el mismo patrón de logos en cada cara.



Después de texturizar el modelo usando UV Editing, lo exporté como .obj y lo agregué al código de OpenGL.

```
Model Holocron_M;
```

```
Holocron_M = Model();  
Holocron_M.LoadModel("Models/holocron_simple.obj");
```

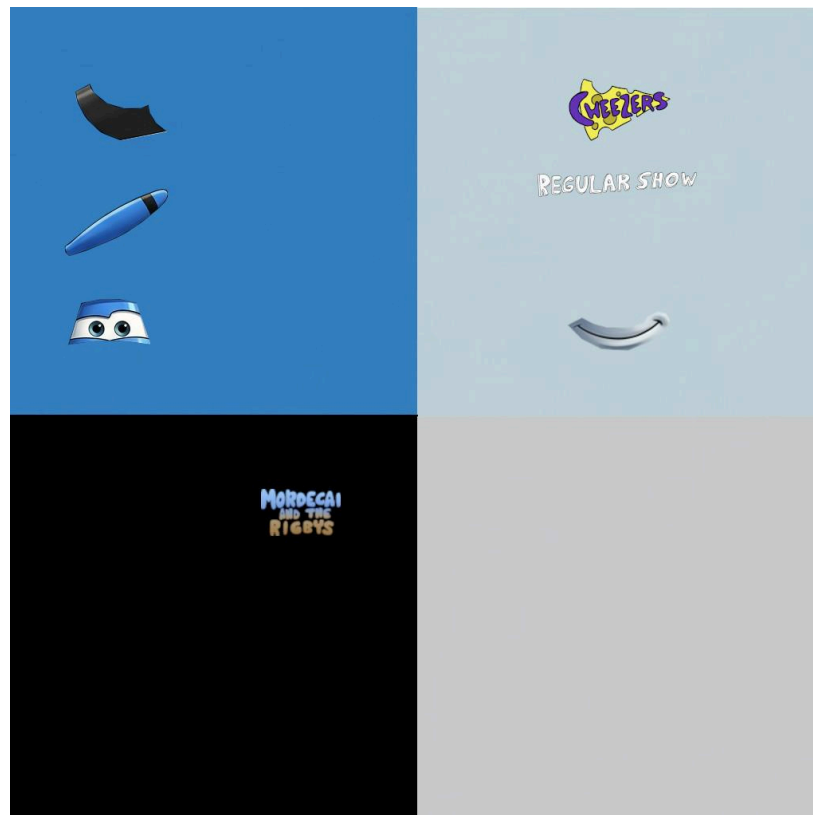
Y posteriormente lo dibujo también, modificando su posición para que no se traslape con el anterior holocrón y agregando un color rojo que lo diferencie.

```
// Holocron Blender  
color = glm::vec3(1.0f, 0.0f, 0.0f); //rojo  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(-7.0f, 5.0f, 0.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));  
model = glm::rotate(model, 180.0f * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Holocron_M.RenderModel();
```

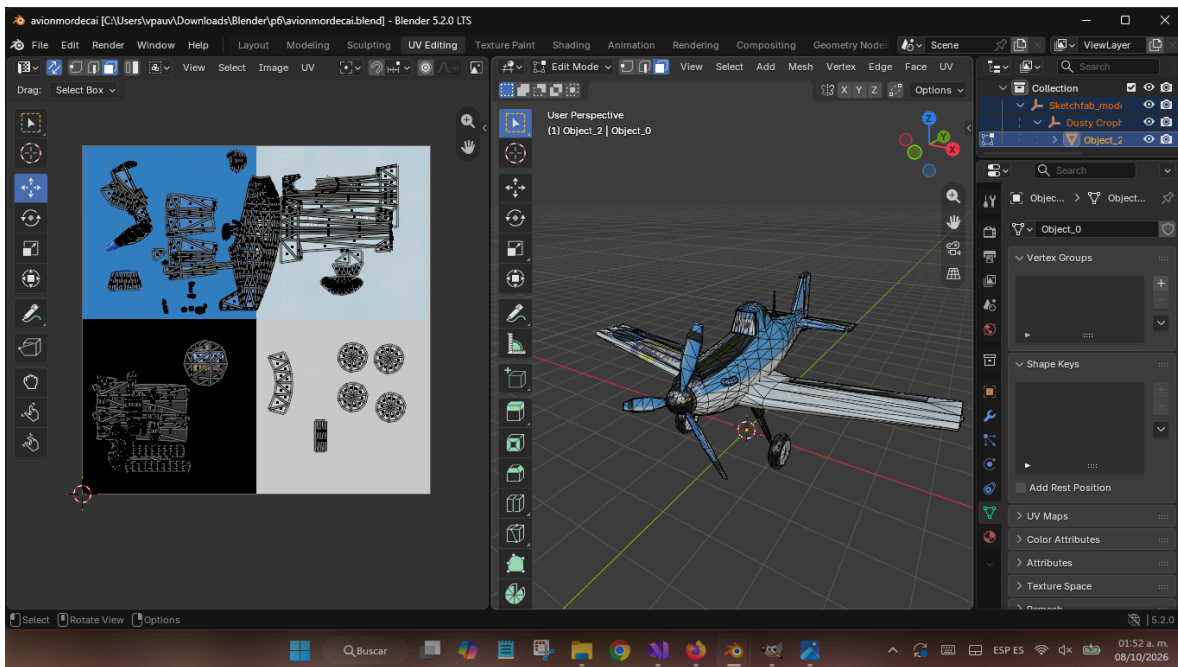
Finalmente para el último ejercicio utilizo Blender nuevamente para texturizar el modelo de un avión con las imágenes proporcionadas por la IA de mi personaje con el estilo de la película de Aviones:



Para esto realizo una imagen que contendrá cada detalle de ojos, boca, nariz, logos y demás detalles como las hélices y elementos adicionales para la cola del avión que recuerdan al diseño de mi personaje, todo en una imagen con los 4 colores principales del modelo y en un tamaño de 1024 x 1024.



Posteriormente texturizo en UV Editing el modelo y lo exporto como .obj.



Lo agrego al código de OpenGL:

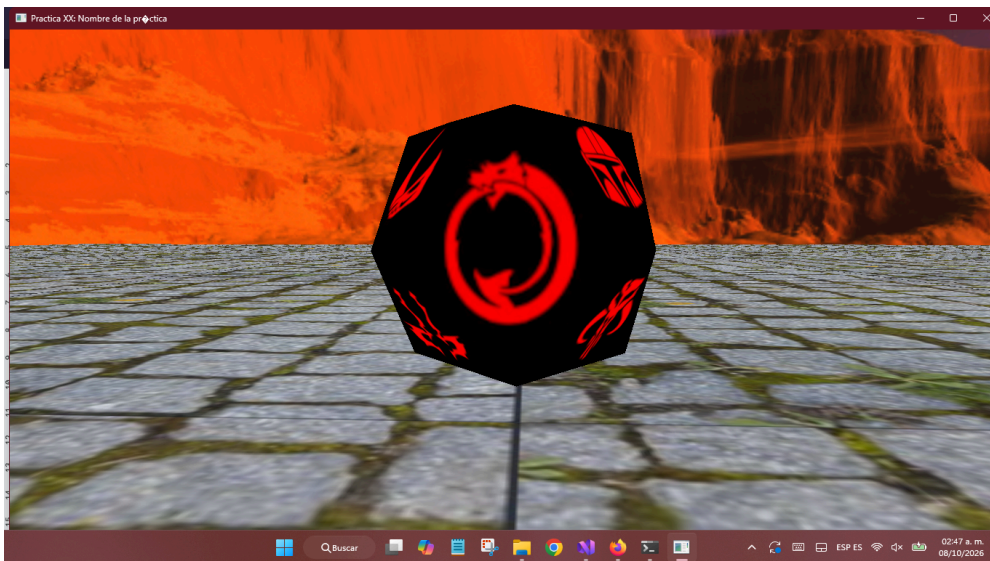
```
Model Avion_M;
```

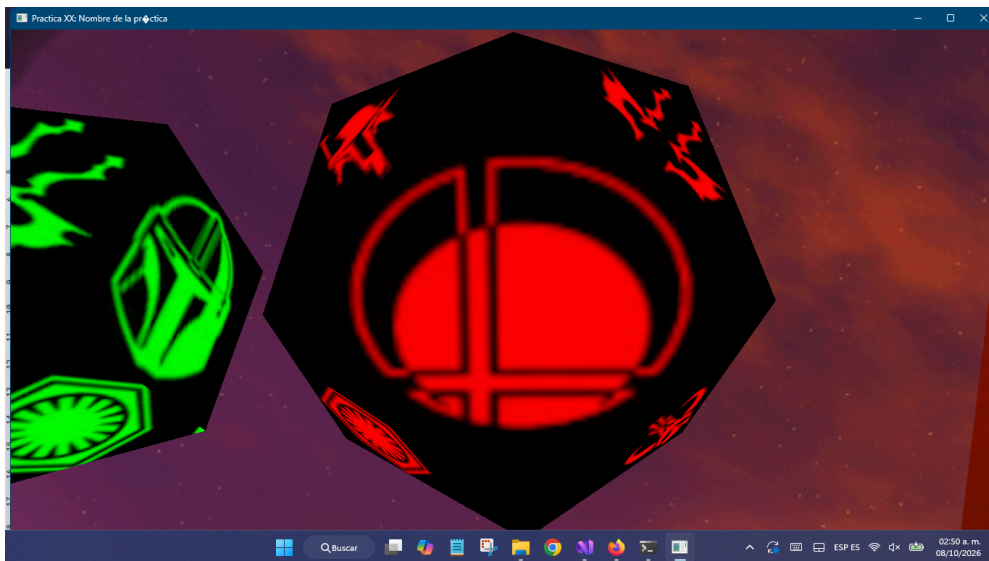
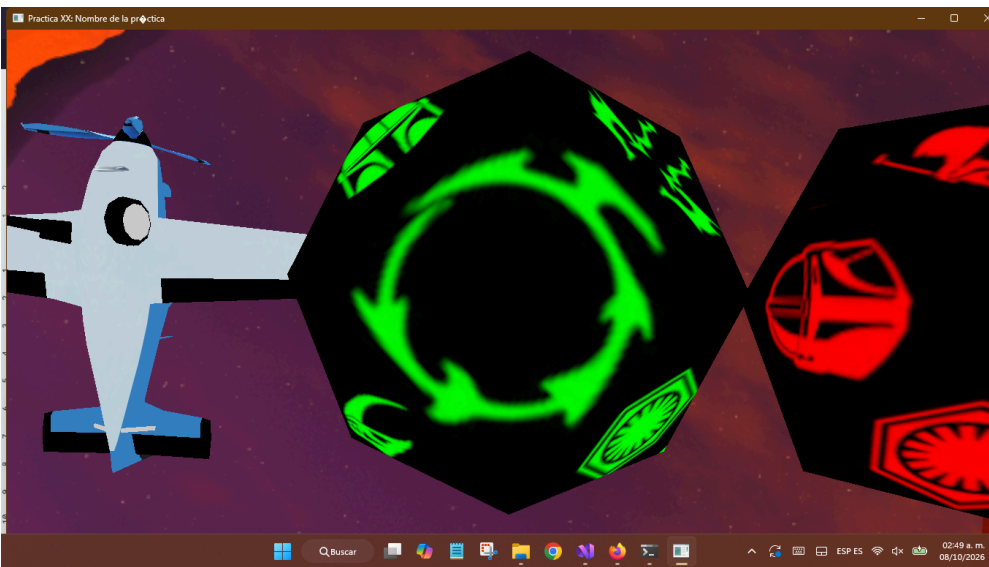
```
Avion_M = Model();  
Avion_M.LoadModel("Models/avion_mordecai.obj");
```

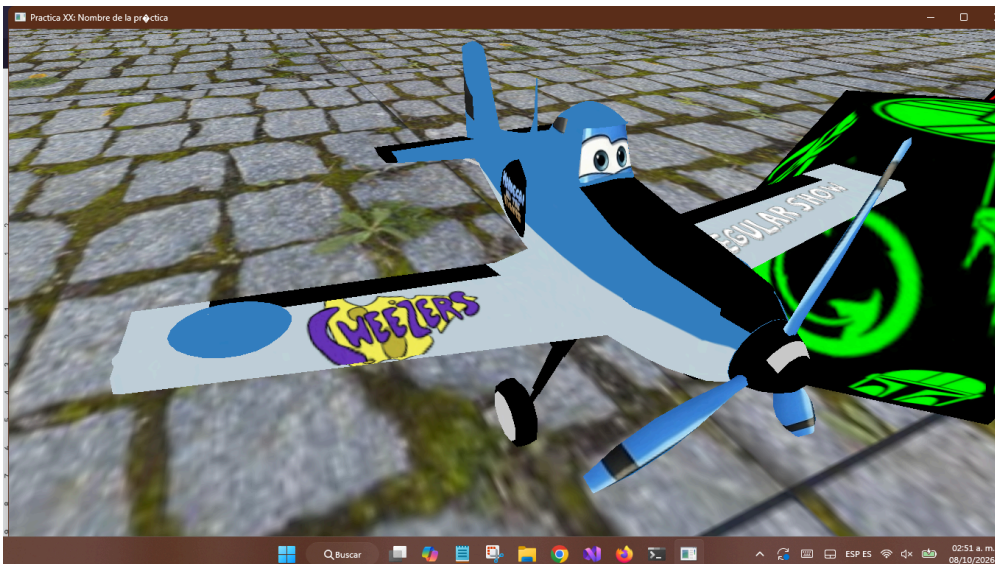
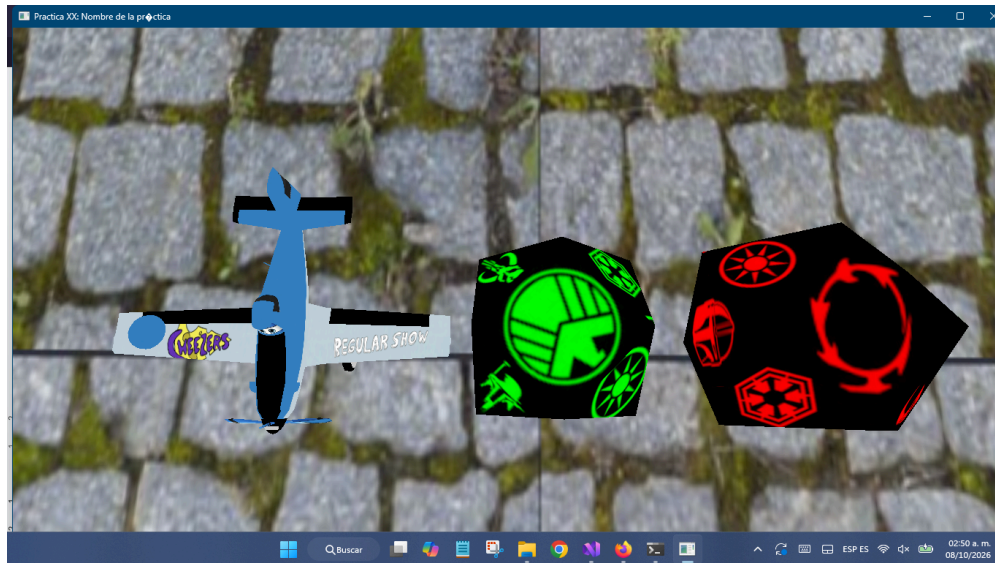
Y por último dibujo el modelo junto con los holocrones de la siguiente manera:

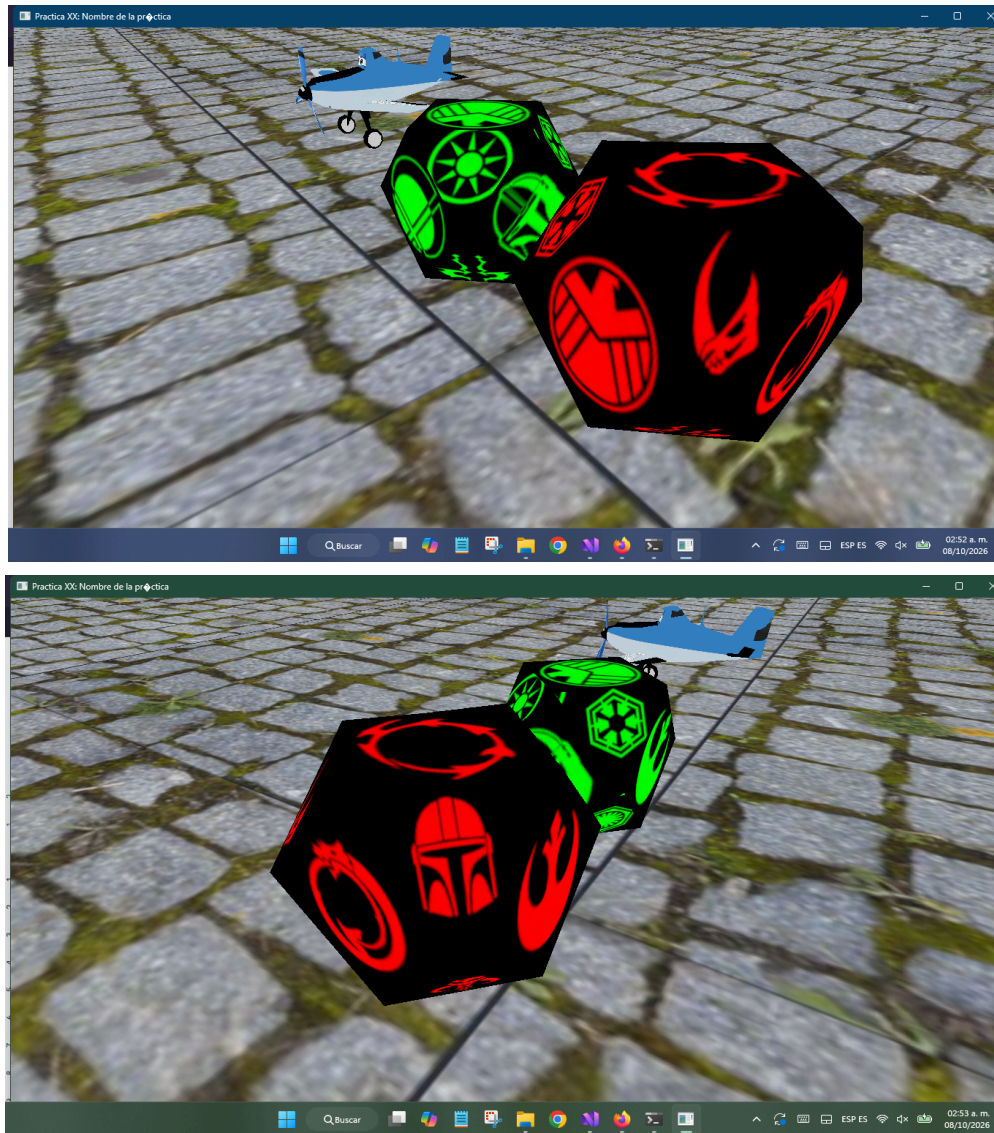
```
// Avion Blender  
color = glm::vec3(1.0f, 1.0f, 1.0f); // blanco  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
model = glm::mat4(1.0);  
model = glm::translate(model, glm::vec3(7.0f, 5.0f, 0.0f));  
model = glm::scale(model, glm::vec3(1.0f, 1.0f, 1.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
Avion_M.RenderModel();
```

La ejecución final, con todos los modelos se muestra a continuación:









2. Problemas presentados.

No hubo ningún problema en la solución de la práctica, además del manejo de los ejes por medio de código.

3. Conclusión:

Con la realización de esta práctica pude comprender mejor el proceso de texturizado de modelos tridimensionales y la importancia de utilizar correctamente las coordenadas de textura para obtener los resultados deseados. El uso de herramientas como GIMP, GeoGebra y Blender permitió trabajar con diferentes métodos de texturizado, tanto mediante la asignación de coordenadas directamente en el código como a través de UV Editing. Aunque el manejo de las coordenadas en OpenGL requirió mayor atención para posicionar correctamente las imágenes, los resultados fueron satisfactorios.

